

TEAM 7 – E-WASTE BATTERY EXTRACTION

CV-15: Bounding Box & Centroid Publisher

Technical Implementation & Verification Documentation

Student Name	Sarvesh Venkatesh Vidya
Student ID	S225162255
Unit	SIT782 – Capstone Team Project (B)
Project	Redback Operations - E-Waste Robotic Battery Extraction
Sub-team	Computer Vision
Sprint	Sprint 2
Task	CV-15 – Bounding Box & Centroid Publisher
Technology	ROS 2 Jazzy, Python, YOLO11n-seg / ONNX Runtime

Individual technical contribution record

1. Objective

The objective of CV-15 was to convert the post-processed computer-vision detection results into the agreed ROS 2 detection message and publish one detection collection for each processed camera frame. The message provides the detected class, confidence, bounding-box information, four ordered bounding-box corners, and the segmentation-mask centroid.

2. Implementation

2.1 ROS 2 Detection Message

The PhoneDetection ROS 2 message was extended to represent the information required by downstream computer-vision and robotics components. Four ordered pixel-space corners and the segmentation centroid were added alongside the existing class, confidence, axis-aligned bounding box, oriented bounding-box and world-position fields.

Field / Group	Purpose	Convention
class_id	Detected object class	Model class identifier
score	Detection confidence	0–1
bbox_x1 ... bbox_y2	Axis-aligned bounding box	Original image pixels
bbox_corner_1 ... 4	Four oriented-box corners	Top-left → top-right → bottom-right → bottom-left
centroid_x / centroid_y	Segmentation-mask centroid	Pixel coordinates
mar_*	Minimum-area rectangle information	Pixel coordinates

2.2 Centroid Calculation

The inference post-processing calculates the centroid from the cleaned segmentation mask. The mean x and y coordinates of the mask pixels are used as the centroid. If no mask pixels are available, the centre of the axis-aligned bounding box is used as a fallback.

2.3 Ordered Bounding-Box Corners

The oriented bounding-box corners generated from the minimum-area rectangle are transferred into the ROS 2 message using a consistent order: top-left, top-right, bottom-right, and bottom-left. This provides a predictable convention for later visualisation and localisation.

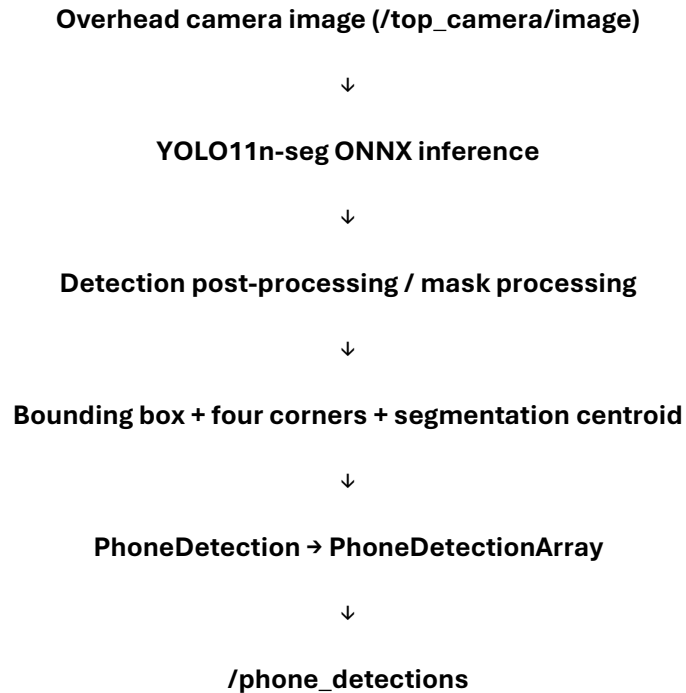
2.4 ROS 2 Publisher

The detector node populates a PhoneDetection message for each post-processed detection and places the messages into a PhoneDetectionArray. The collection is published on the agreed /phone_detections topic. When no object is detected, an empty detection collection is still published.

2.5 Coordinate System

Pixel coordinates are represented in the original overhead-camera image space. The current overhead camera resolution is 1280 × 960 pixels.

3. Processing Flow

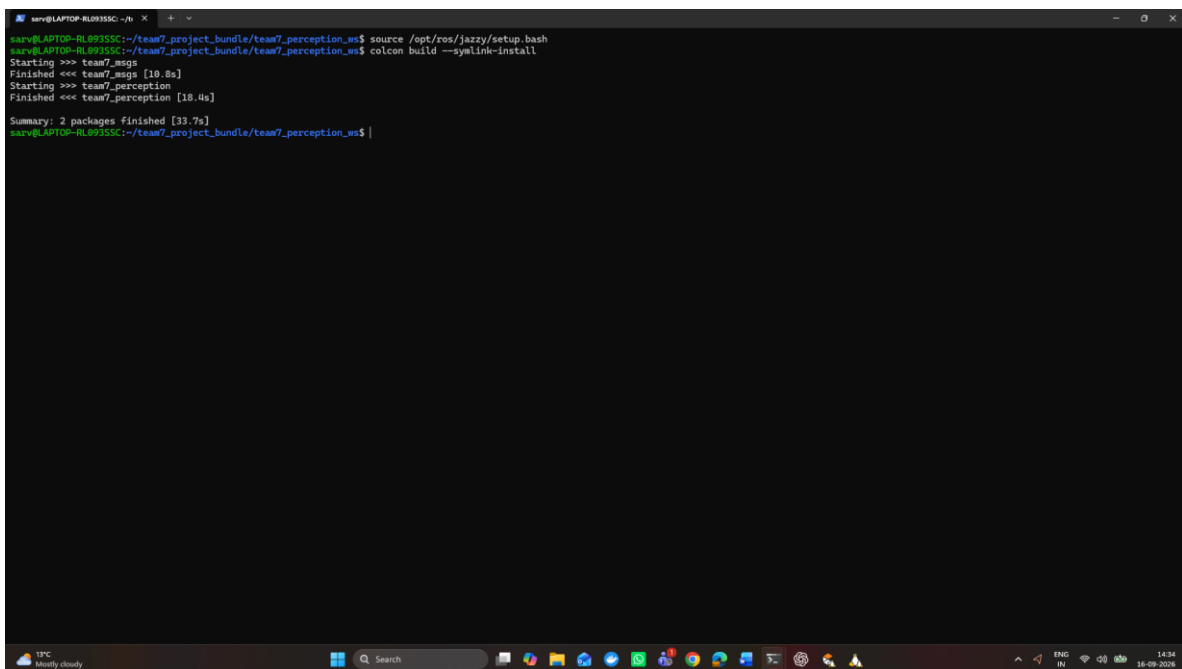


4. Verification and Testing

The implementation was verified at build, interface, topic, and runtime levels.

4.1 Workspace Build

The ROS 2 workspace was rebuilt using colcon with symlink installation. Both team7_msgs and team7_perception completed successfully.



```
karv@LAPTOP-RL9355C: ~/team7_project_bundle/team7_perception_ws$ source /opt/ros/jazzy/setup.bash
karv@LAPTOP-RL9355C: ~/team7_project_bundle/team7_perception_ws$ colcon build --symlink-install
Starting >>> team7_msgs
Finished <<< team7_msgs [10.8s]
Starting >>> team7_perception
Finished <<< team7_perception [10.4s]
Summary: 2 packages finished [33.7s]
karv@LAPTOP-RL9355C: ~/team7_project_bundle/team7_perception_ws$
```

Figure 1 – Successful ROS 2 Workspace Build

4.2 ROS 2 Message Interface

The generated PhoneDetection interface was inspected using `ros2 interface show team7\_msgs/msg/PhoneDetection`. The output confirmed the four corner coordinate pairs and centroid_x / centroid_y fields.

```

sarv@LAPTOP-RL8935SC:~/team7_project_bundle/team7_perception_ws$ colcon build --symlink-install
Starting >>> team7_msgs
Finished <<< team7_msgs [10.8s]
Starting >>> team7_perception
Finished <<< team7_perception [18.4s]

Summary: 2 packages finished [33.7s]
sarv@LAPTOP-RL8935SC:~/team7_project_bundle/team7_perception_ws$ source install/setup.bash
ros2 interface show team7_msgs/msg/PhoneDetection
std_msgs/Header header
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id
string class_id
float64 score

# Axis-aligned bounding box in original /top_camera/image coordinates (1280x960)
int32 bbox_x1
int32 bbox_y1
int32 bbox_x2
int32 bbox_y2

# Four ordered corners of the oriented bounding box.
# Order: top-left, top-right, bottom-right, bottom-left.
float64 bbox_corner_1_x
float64 bbox_corner_1_y
float64 bbox_corner_2_x
float64 bbox_corner_2_y
float64 bbox_corner_3_x
float64 bbox_corner_3_y
float64 bbox_corner_4_x
float64 bbox_corner_4_y

# Centroid of the segmentation mask in pixel coordinates
float64 centroid_x
float64 centroid_y

# Oriented box from the segmentation mask
float64 mar_center_x
float64 mar_center_y
float64 mar_width
float64 mar_height
float64 angle_deg

# Projected world-frame position
float64 world_x
float64 world_y
float64 world_z
bool world_valid
sarv@LAPTOP-RL8935SC:~/team7_project_bundle/team7_perception_ws$
```

Figure 2 – PhoneDetection ROS 2 Interface

4.3 Detection Topic

The detector node was launched and the /phone_detections topic was inspected to confirm that the detection publisher was active.

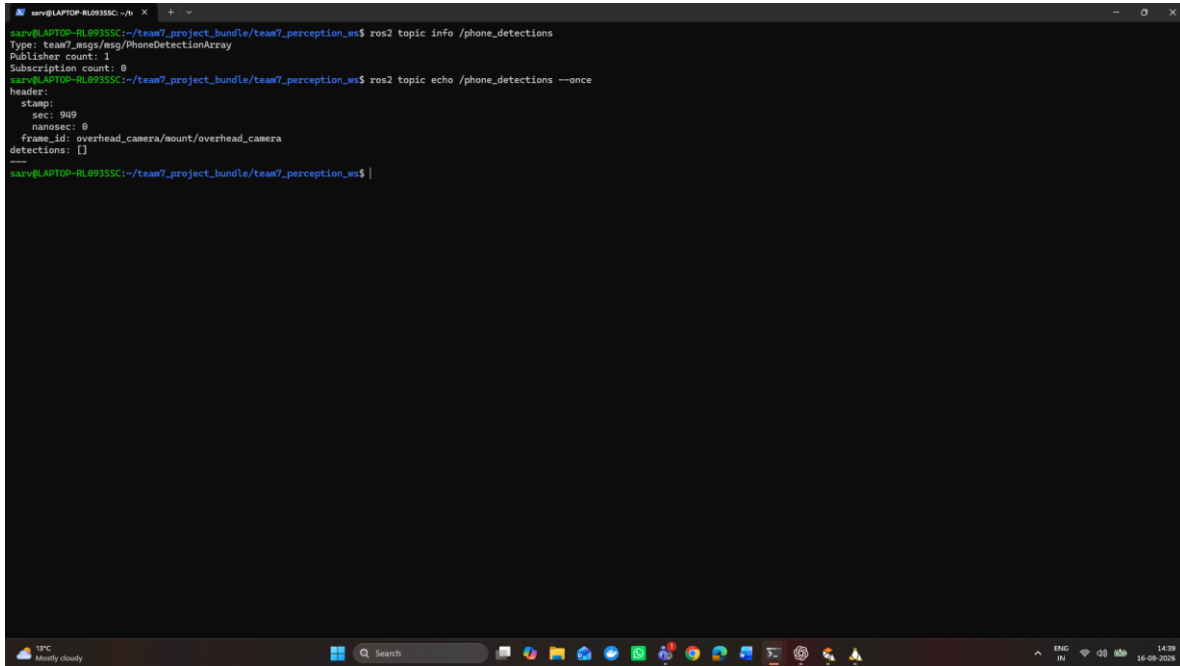
```

sarv@LAPTOP-RL8935SC:~/team7_project_bundle/team7_perception_ws$ ros2 topic info /phone_detections
Type: team7_msgs/msg/PhoneDetectionArray
Publisher count: 1
Subscription count: 0
sarv@LAPTOP-RL8935SC:~/team7_project_bundle/team7_perception_ws$ ros2 topic echo /phone_detections --once
header:
  stamp:
    sec: 989
    nanosec: 0
  frame_id: overhead_camera/mount/overhead_camera
  detections: []
---
sarv@LAPTOP-RL8935SC:~/team7_project_bundle/team7_perception_ws$
```

Figure 3 – /phone_detections Topic Information

4.4 Runtime Detection Collection

A live message was retrieved from /phone_detections using `ros2 topic echo /phone_detections --once`. During this test frame, the model did not produce an object detection, and the node correctly published an empty detection collection (`detections: []`). This verifies the required no-detection behaviour. A populated detection will be further validated during later visualisation and end-to-end testing when a valid model detection is available.



```
marv@LAPTOP-RL0935SC:~/team7_project_bundle/team7_perception_ws$ ros2 topic info /phone_detections
Type: team7_msgs/msg/PhoneDetectionArray
Publisher count: 1
Subscription count: 0
marv@LAPTOP-RL0935SC:~/team7_project_bundle/team7_perception_ws$ ros2 topic echo /phone_detections --once
header:
  stamp:
    sec: 949
    nanosec: 0
  frame_id: overhead_camera/mount/overhead_camera
detections: []
---
```

Figure 4 – Runtime PhoneDetectionArray Output

5. Technical Outcome

CV-15 was implemented and verified as a ROS 2 message-publishing component. The detection message now provides the information required for subsequent visualisation and localisation work, including four consistently ordered bounding-box corners and a segmentation-mask centroid. The publisher also maintains a valid detection collection when no object is detected.

6. Current Validation Limitation

The live simulation verification performed for this task produced an empty detection array because the current inference run did not return a valid object detection. Therefore, this evidence confirms the publisher and no-detection path, but does not claim a populated corner/centroid example. Populated detection values can be validated during CV-16 annotated visualisation and subsequent end-to-end testing.

7. Task Status

CV-15 – COMPLETED

Implementation, workspace build, ROS 2 interface verification, topic publication, and empty detection collection behaviour have been completed and documented.